

Cross Validation Lab 4

Dr. Luisa Barbanti & Dr. Matteo Tanadini

Applied Machine Learning and Predictive Modelling 1, HS25 (HSLU)

Contents

1	Overfit	2
2	Assessing the predictive performance of a model: cross validation	4
2.1	In-sample performance	4
2.2	Out-of-sample performance	6
2.2.1	Randomness of the splitting procedure	9
3	Typical cross validation schemes	12
4	Take home messages	15
5	Appendix	16
5.1	What is “good”?	16
	Session Information	17

1 Overfit

When fitting a model to a given dataset we face questions such as “what model do I need to fit to these data? How complex should it be? Which predictors shall I include in the model?”.

Let’s fit a GLM Poisson model to the *esdcomp* dataset found in the *{faraway}* package. This dataset is about the number of complaints received by 44 doctors working in an emergency service at a hospital (type *?esdcomp* for more information).

Shall we really include all predictors? How complex should the model be (e.g. shall I include non-linear effects)? What interactions should be there?

Let’s load the data and inspect the first few observations.

```
library(faraway)
data("esdcomp")
```

```
head(esdcomp)
```

	visits	complaints	residency	sex	revenue	hours
1	2014	2	Y	F	263	1287
2	3091	3	N	M	335	1588
3	879	1	Y	M	206	705
4	1780	1	N	M	226	1006
5	3646	11	N	M	289	1667
6	2690	1	N	M	276	1518

We fit a Poisson Model to these data. Here we fit a model that implies the presence of an interaction between *sex* and any other predictor in the model (i.e. the effect of all predictors is assumed to differ between females and males). In addition, we also assume that all continuous predictors can have a non-linear (cubic) effect.

```
glm.complaints.1 <- glm(complaints ~ (poly(visits, degree = 3) +
                                residency +
                                poly(revenue, degree = 3) +
                                poly(hours, degree = 3)) * sex,
                        family = "poisson",
                        data = esdcomp)
```

Let’s look at the summary output.

```
summary(glm.complaints.1)
```

Call:

```
glm(formula = complaints ~ (poly(visits, degree = 3) + residency +
    poly(revenue, degree = 3) + poly(hours, degree = 3)) * sex,
    family = "poisson", data = esdcomp)
```

Coefficients:

	Estimate	Std. Error	z value	Pr(> z)
(Intercept)	5.27	3.64	1.45	0.15
poly(visits, degree = 3)1	92.36	69.42	1.33	0.18
poly(visits, degree = 3)2	-36.33	39.93	-0.91	0.36

poly(visits, degree = 3)3	39.71	27.39	1.45	0.15
residencyY	-2.25	1.99	-1.13	0.26
poly(revenue, degree = 3)1	-12.01	28.42	-0.42	0.67
poly(revenue, degree = 3)2	17.48	25.88	0.68	0.50
poly(revenue, degree = 3)3	2.17	13.23	0.16	0.87
poly(hours, degree = 3)1	-88.15	74.80	-1.18	0.24
poly(hours, degree = 3)2	26.03	22.88	1.14	0.26
poly(hours, degree = 3)3	-9.92	6.81	-1.46	0.15
sexM	-4.09	3.64	-1.12	0.26
poly(visits, degree = 3)1:sexM	-92.36	69.50	-1.33	0.18
poly(visits, degree = 3)2:sexM	37.20	40.00	0.93	0.35
poly(visits, degree = 3)3:sexM	-39.30	27.42	-1.43	0.15
residencyY:sexM	2.10	2.01	1.04	0.30
poly(revenue, degree = 3)1:sexM	12.36	28.45	0.43	0.66
poly(revenue, degree = 3)2:sexM	-18.25	25.89	-0.70	0.48
poly(revenue, degree = 3)3:sexM	-3.01	13.25	-0.23	0.82
poly(hours, degree = 3)1:sexM	91.91	74.88	1.23	0.22
poly(hours, degree = 3)2:sexM	-26.36	23.07	-1.14	0.25
poly(hours, degree = 3)3:sexM	9.48	7.10	1.34	0.18

(Dispersion parameter for poisson family taken to be 1)

Null deviance: 89.447 on 43 degrees of freedom
 Residual deviance: 31.380 on 22 degrees of freedom
 AIC: 198.2

Number of Fisher Scoring iterations: 5

What is striking here is the very large number of parameters. Indeed, in this model there are 22 parameters estimated from 44 observations only.

```
( N <- nrow(esdcomp) )
```

```
[1] 44
```

```
( p <- length(coef(glm.complaints.1)))
```

```
[1] 22
```

As we are fitting a very complex model, we may think that we have a good fit of the data. However, the issue here is that **we may also model noise in the data. This is called overfit.**

Obviously, the more flexibility we allow, the higher the chance to overfit the data. However, note that the opposite concept exists as well: underfitting. The latter involves fitting a too simple model to data (e.g. fitting a simple regression line to complex data).

The obvious question that naturally raises here is: “how to select the best model?”

We realise that our model is overfitting (underfitting) the data when we use it to make predictions on **new data** (“new data” refers to data that were not used to train the model). Generally, a model that performs well on new data is a model that does not overfit our dataset at hand.

However, we do not have access to new/future data, we just have the dataset at hand. Thus, we can only **estimate the generalization performance of a model on new data**. Based on this estimate we can then select the “best” model.

In the very **specific case of prediction**, a possible approach to quantify overfit is to “simulate” the situation where we have a test data set. The idea is to not use all the observations to fit the model and quantify overfit on the “left-out” part of the data. This way to proceed is called **cross validation (CV)**.

2 Assessing the predictive performance of a model: cross validation

Let’s put in practice the principle we just discussed. We split the data and use one part to fit a model (i.e. **training** data) and then use the other part of the data to test whether the model is able to make good predictions (i.e. **testing** data).

We use the tree growth dataset of Lecture 2 again. Let’s get the data.

```
d.trees <- read.csv2("../ ../ ../ Datasets/TreesChamagne2017_Lab.csv",
                     stringsAsFactors = TRUE)
str(d.trees)
```

```
'data.frame':  557 obs. of  12 variables:
 $ growth.rate      : num  0.702 1.139 1.394 1 1.355 ...
 $ species          : Factor w/ 4 levels "Fagus","Larix",...: 1 1 1 3 3 3 1 1 1 1 ...
 $ site            : int   1 1 12 12 12 12 12 12 12 12 ...
 $ Density.tree.Class: Factor w/ 3 levels "high","low","medium": 1 2 3 3 1 1 2 3 2 3 ...
 $ age             : num   55 42 60 63 58 57 65 61 65 58 ...
 $ size            : num  23296 33718 57300 67297 97764 ...
 $ density.site     : num   43.6 43.6 35.8 35.8 35.8 ...
 $ density.tree     : num   6.03 3.91 5.54 5.06 5.89 ...
 $ diversity.tree   : num   1 1.14 1.98 2.67 2.19 ...
 $ diversity.site   : num   1.28 1.28 2.27 2.27 2.27 ...
 $ sp.richness      : int   1 1 2 2 2 2 2 2 2 2 ...
 $ SiteID          : int   1 1 12 12 12 12 12 12 12 12 ...
```

Assume we are looking for a model that is **good at making predictions**. To find the “best” one, we **should compare the performance of many models**. Here for didactic reasons we compare two models only.

2.1 In-sample performance

To start with, we fit the two models on all observations and look at model performance. The performance measured on models fitted to all data available is called “in-sample” performance.

```
## 1) the "simple" model
lm.trees.1 <- lm(growth.rate ~ species + poly(density.site, degree = 2),
                 data = d.trees)

##
## 2) the "complex" model
lm.trees.2 <- lm(growth.rate ~ species * (poly(age, degree = 3) +
                                           poly(diversity.site, degree = 3) +
                                           poly(density.site, degree = 3)),
                 data = d.trees)
```

Among the many possible choices of performance measures, we arbitrarily chose the R-squared. We know that the R-squared always increases when extra predictor variables are added to the model. So we expect that the R-squared is higher for the more complex model, and this is indeed the case:

```
summary(lm.trees.1)$r.squared
```

```
[1] 0.18
```

```
##  
summary(lm.trees.2)$r.squared
```

```
[1] 0.29
```

The more complex model explains way more of the variance (i.e. 29% vs. 18%).

We also have a look at the adjusted R-squared. The adjusted R-squared does not automatically increase with more predictors by penalizing a higher number of predictors.

```
summary(lm.trees.1)$adj.r.squared
```

```
[1] 0.17
```

```
##  
summary(lm.trees.2)$adj.r.squared
```

```
[1] 0.24
```

The two models are more similar when comparing the adjusted R-squared. However, also when penalizing more predictors by using the adjusted R-squared, the more complex model still explains more of the variance (i.e. 24% vs. 17%).

We also have a look at the mean squared error (MSE) of both models.

```
mean((d.trees$growth.rate - predict(lm.trees.1))^2)
```

```
[1] 0.064
```

```
##  
mean((d.trees$growth.rate - predict(lm.trees.2))^2)
```

```
[1] 0.055
```

Also in terms of MSE, the more complex model seems better.

But the question that we want to address now is whether this model is also better at making predictions **on new observations that have not been used to train the model**.

As mentioned above, we now use a part of the data to train the model and then test its predictive performance on a part of data that was left aside. There are many ways how we could split the data. Here, for the sake of simplicity, we start by splitting the data into two equally-sized parts (also sometimes called the “Validation set approach”).

```
## Some new commands we will use:
( folds <- cut(1:25, breaks = 6, labels = FALSE) )
```

[1] 1 1 1 1 1 2 2 2 2 3 3 3 3 4 4 4 4 5 5 5 5 6 6 6 6

```
##  
which(folds == 2)
```

```
[1] 6 7 8 9
```

The function `cut()` helps us split the data into folds that are of equal size if possible. Here, we split our data into two folds, the train sample and test sample. We want them to be of equal size, where we have

```
nrow(d.trees)
```

[1] 557

many observations. Since the number of observations in the dataset is odd, we will have one more data point in either train or test sample (which does not matter).

We now create the two halves of the actual data with the function *cut()*.

```
( folds <- cut(1:nrow(d.trees), breaks = 2, labels = FALSE) )
```

[illegible]

```
## look at the number of observations in each fold
table(folds)
```

```
folds
  1  2
279 278
```

We will now split the data according to these two folds, that is we obtain the “train” sample and the “test” sample. We will then fit the model with the train data and then test the model on the test data. We do that for the “simple” model first.

```
## 1) prepare data
( ind.test <- which(folds == 1) )
```

```
[1]  1  2  3  4  5  6  7  8  9 10 11 12 13 14 15 16 17 18
[19] 19 20 21 22 23 24 25 26 27 28 29 30 31 32 33 34 35 36
[37] 37 38 39 40 41 42 43 44 45 46 47 48 49 50 51 52 53 54
[55] 55 56 57 58 59 60 61 62 63 64 65 66 67 68 69 70 71 72
[73] 73 74 75 76 77 78 79 80 81 82 83 84 85 86 87 88 89 90
[91] 91 92 93 94 95 96 97 98 99 100 101 102 103 104 105 106 107 108
[109] 109 110 111 112 113 114 115 116 117 118 119 120 121 122 123 124 125 126
[127] 127 128 129 130 131 132 133 134 135 136 137 138 139 140 141 142 143 144
[145] 145 146 147 148 149 150 151 152 153 154 155 156 157 158 159 160 161 162
[163] 163 164 165 166 167 168 169 170 171 172 173 174 175 176 177 178 179 180
[181] 181 182 183 184 185 186 187 188 189 190 191 192 193 194 195 196 197 198
[199] 199 200 201 202 203 204 205 206 207 208 209 210 211 212 213 214 215 216
[217] 217 218 219 220 221 222 223 224 225 226 227 228 229 230 231 232 233 234
[235] 235 236 237 238 239 240 241 242 243 244 245 246 247 248 249 250 251 252
[253] 253 254 255 256 257 258 259 260 261 262 263 264 265 266 267 268 269 270
[271] 271 272 273 274 275 276 277 278 279
```

```
d.trees.test <- d.trees[ind.test, ]
d.trees.train <- d.trees[- ind.test, ]
##
nrow(d.trees.test)
```

```
[1] 279
```

```
nrow(d.trees.train)
```

```
[1] 278
```

```
##
## 2) fit the model with "train" data
lm.trees.1.train <- lm(formula = formula(lm.trees.1),
                       data = d.trees.train)
##
## 3) make prediction on the test data
predicted.trees.1.test <- predict(lm.trees.1.train,
                                 newdata = d.trees.test)
##
## 4) compute MSE on the test data
mean((d.trees.test$growth.rate - predicted.trees.1.test)^2)
```

```
[1] 0.07
```

So the R-MSE for the simpler model on the out-of-sample data is 0.07. Let's repeat the same procedure with the "complex" model.

```
## 2) fit the model with "train" data
lm.trees.2.train <- lm(formula = formula(lm.trees.2),
                        data = d.trees.train)

##
## 3) make prediction on the test data
predicted.trees.2.test <- predict(lm.trees.2.train,
                                  newdata = d.trees.test)

##
## 4) compute MSE on the test data
mean((d.trees.test$growth.rate - predicted.trees.2.test)^2)
```

```
[1] 49
```

The MSE for the complex model measured on the test data is actually much larger than before: 49.4.

Based on these results, we would conclude that the simpler model is better suited to make predictions as it does not overfit the data.

Question: *Would you trust these results, or is there anything else you would add to make the results more "robust" and credible?*

2.2.1 Randomness of the splitting procedure

To answer the previous question ask yourself the following questions: “What happens if another split is used? Do we get the same results? Maybe, just by chance I computed the out-of-sample MSE on a split that is particularly advantageous to the simpler model?!”

Let’s repeat the splitting procedure several times to get more robust results. To perform the random split into two halves, we will use the following code. Note that we first permute the rows of our dataset and then use again the *cut()*-function to define the two halves.

```
## randomly permute the row-numbers:
set.seed(1)
n <- nrow(d.trees)
## permuted row-numbers (= indices)
( inds.permuted <- sample(1:n, replace = FALSE) )
```

```
[1] 129 509 471 299 270 187 307 277 494 330 37 105 485 382 326 548 422 111
[19] 404 532 506 343 121 40 375 248 198 378 39 435 390 280 45 402 22 193
[37] 371 499 104 543 492 465 176 345 110 84 359 29 141 252 406 221 412 108
[55] 304 33 525 149 287 102 145 329 487 536 339 118 346 503 107 64 478 224
[73] 103 316 51 537 290 138 557 554 443 282 143 442 285 408 170 48 204 295
[91] 24 181 214 296 225 493 491 163 43 1 510 78 507 518 284 116 233 293
[109] 61 430 369 86 327 513 355 551 300 49 361 484 242 246 305 306 247 239
[127] 219 135 540 20 377 472 483 53 400 356 130 490 65 441 511 546 124 77
[145] 218 98 194 19 273 31 477 174 403 237 75 16 534 265 434 92 122 152
[163] 480 207 249 160 229 140 126 14 452 524 271 421 419 358 409 212 127 133
[181] 41 364 373 328 36 528 297 488 479 309 380 362 344 117 368 498 50 336
[199] 286 254 349 72 456 168 374 387 113 234 535 73 27 427 15 294 62 132
[217] 35 414 416 185 445 153 426 547 255 231 28 549 148 473 379 298 60 453
[235] 335 268 93 418 202 505 476 241 502 342 376 446 217 526 450 504 385 553
[253] 432 209 97 486 541 424 56 332 199 256 201 405 2 352 131 451 474 156
[271] 197 220 235 458 173 83 208 431 338 545 253 180 162 482 460 167 440 190
[289] 161 556 433 391 275 495 25 81 222 203 519 3 179 147 269 128 276 337
[307] 4 205 325 383 192 232 466 530 106 172 341 357 23 157 313 438 164 468
[325] 119 99 381 496 150 240 169 312 211 308 18 44 272 457 91 353 32 425
[343] 481 420 216 125 550 89 350 500 21 79 115 101 267 407 71 544 59 90
[361] 158 58 514 114 370 134 34 394 324 17 542 539 501 354 66 461 228 136
[379] 263 230 397 5 74 428 264 206 238 363 447 393 69 196 401 322 257 314
[397] 42 123 244 38 413 8 317 47 398 112 213 46 139 288 522 512 250 527
[415] 411 319 340 215 30 26 109 7 396 347 459 226 279 448 260 261 94 10
[433] 281 189 6 188 184 171 54 531 555 96 154 100 200 515 87 520 146 360
[451] 331 12 436 251 497 395 120 258 195 318 444 151 259 464 366 85 302 266
[469] 55 80 399 475 289 523 278 423 283 95 533 166 52 538 365 351 11 392
[487] 470 57 301 144 348 68 9 183 223 292 323 175 178 372 274 67 552 334
[505] 454 186 529 311 333 165 137 236 262 13 469 516 70 462 489 415 159 76
[523] 384 227 417 508 517 429 388 320 367 142 182 386 191 449 177 303 521 321
[541] 467 455 310 155 291 63 243 439 410 88 437 315 463 210 389 245 82
```

```
## we permute the data according to the permuted row-numbers
d.trees.permuted <- d.trees[inds.permuted, ]
dim(d.trees.permuted)
```

```
[1] 557 12
```

```
dim(d.trees)
```

[1] 557 12

```
## we create the 2 folds of (roughly) equal size as before:
( folds <- cut(1:n, breaks = 2, labels = FALSE) )
```

[illegible]

```
##  
( ind.test <- which(folds == 1) )
```

[1]	1	2	3	4	5	6	7	8	9	10	11	12	13	14	15	16	17	18
[19]	19	20	21	22	23	24	25	26	27	28	29	30	31	32	33	34	35	36
[37]	37	38	39	40	41	42	43	44	45	46	47	48	49	50	51	52	53	54
[55]	55	56	57	58	59	60	61	62	63	64	65	66	67	68	69	70	71	72
[73]	73	74	75	76	77	78	79	80	81	82	83	84	85	86	87	88	89	90
[91]	91	92	93	94	95	96	97	98	99	100	101	102	103	104	105	106	107	108
[109]	109	110	111	112	113	114	115	116	117	118	119	120	121	122	123	124	125	126
[127]	127	128	129	130	131	132	133	134	135	136	137	138	139	140	141	142	143	144
[145]	145	146	147	148	149	150	151	152	153	154	155	156	157	158	159	160	161	162
[163]	163	164	165	166	167	168	169	170	171	172	173	174	175	176	177	178	179	180
[181]	181	182	183	184	185	186	187	188	189	190	191	192	193	194	195	196	197	198
[199]	199	200	201	202	203	204	205	206	207	208	209	210	211	212	213	214	215	216
[217]	217	218	219	220	221	222	223	224	225	226	227	228	229	230	231	232	233	234
[235]	235	236	237	238	239	240	241	242	243	244	245	246	247	248	249	250	251	252
[253]	253	254	255	256	257	258	259	260	261	262	263	264	265	266	267	268	269	270
[271]	271	272	273	274	275	276	277	278	279									

```
d.trees.test <- d.trees.permuted[ind.test, ]
d.trees.train <- d.trees.permuted[- ind.test, ]
```

Question: We permute the rows to define the random splits. Can you think of a type of data where the ordering of the rows in the dataset does have a meaning?

We will now repeat this random splitting into two halves 200 times. In each repetition, we fit both models on the training set and predict on the test set.

```

set.seed(22)
##
mse.simple <- c()
mse.complex <- c()
##
for(i in 1:200){
  ## 1) prepare data
  ## randomly permute the rows:
  n <- nrow(d.trees)
  inds.permuted <- sample(1:n, replace = FALSE)
  d.trees.permuted <- d.trees[inds.permuted, ]

  ## create 2 (roughly) equally sized folds:
  folds <- cut(1:n, breaks = 2, labels = FALSE)
  ##
  ind.test <- which(folds == 1)
  d.trees.test <- d.trees.permuted[ind.test, ]
  d.trees.train <- d.trees.permuted[- ind.test, ]
  ##
  ## simple model ##
  ##
  ## 2) fit the model with "train" data
  lm.trees.1.train <- lm(formula = formula(lm.trees.1),
                        data = d.trees.train)
  ##
  ## 3) make prediction on the test data
  predicted.trees.1.test <- predict(lm.trees.1.train,
                                   newdata = d.trees.test)
  ##
  ## 4) compute MSE
  mse.simple[i] <- mean((d.trees.test$growth.rate - predicted.trees.1.test)^2)

  ##
  ## complex model ##
  ##
  ## 2) fit the model with "train" data
  lm.trees.2.train <- lm(formula = formula(lm.trees.2),
                        data = d.trees.train)
  ##
  ## 3) make prediction on the test data
  predicted.trees.2.test <- predict(lm.trees.2.train,
                                   newdata = d.trees.test)
  ##
  ## 4) compute MSE
  mse.complex[i] <- mean((d.trees.test$growth.rate - predicted.trees.2.test)^2)
}

```

We can now compare the mean MSE over the 200 repetitions for the two models.

```
mean(mse.simple)
```

```
[1] 0.066
```

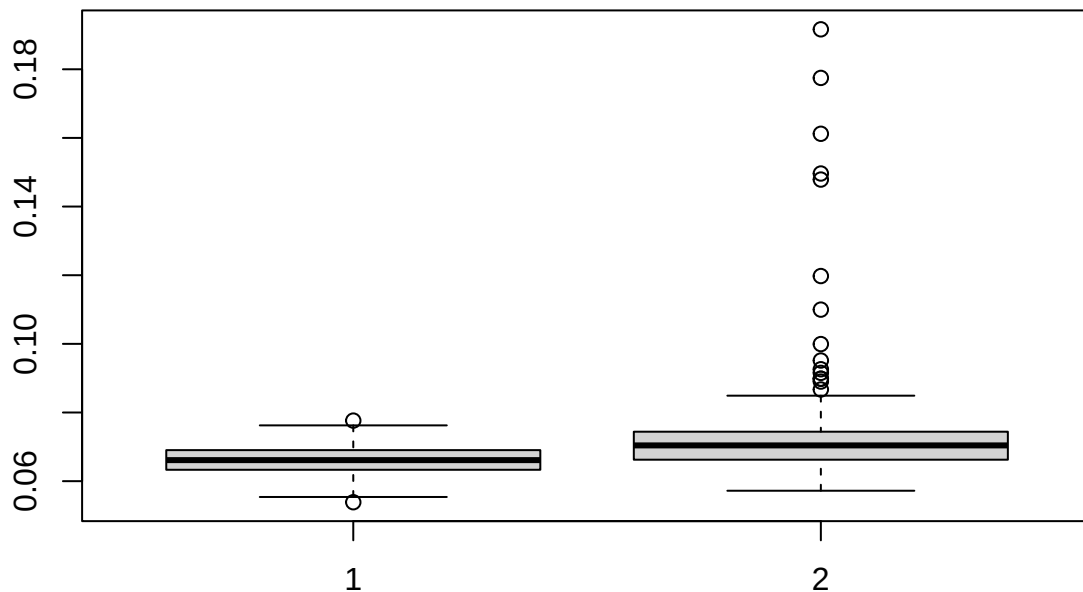
```
mean(mse.complex)
```

```
[1] 0.074
```

Actually, we now see that the two MSE-values are quite similar.

We can also visualise the results.

```
boxplot(mse.simple, mse.complex)
```



These boxplots clearly show that there is some variation in the 200 MSE-values we obtained for the two models. This implies that doing only one split of the data (or very few) is not “robust”. Indeed, we may draw wrong conclusions about which model is better at making predictions.

Therefore, when applying cross validation, several splits are used. Results based on a single split are not “robust”.

Note that we could also use different performance measures (AIC etc). In classification, that is if our response is binary, we also need to adapt our performance measure (e.g. accuracy, AUC, ...).

3 Typical cross validation schemes

There are many ways how data can be split into training and test datasets. Above we used a 50:50 split. Note, that we did not flip the training and test sets, that is we only used the first half as training set and the second as test and we did not fit another model that uses the second as training set and the first as test set. Thus, we did not get a prediction for each of the observations in the data set.

Frequently used approaches obtain a prediction for each observation. Common choices are:

- 10-fold CV (90% of the data are used to train the model and 10% to test it). Then you use another 10% to test and the remaining 90% to train (repeat 10 times in total).
- K-fold CV (for a general K, another typical value is $K = 5$)
- leave-one-out CV (LOOCV, all observations but one are used to train the model)

- ...

Question: *How many models are fitted with leave-one-out CV?*

Question: *If you repeated 10-fold CV 20 times with random folds, would you get 20 different estimated predictive performances?*

Question: *How could we get more stable results with 10-fold CV?*

The choice of the method used depends on the situation. For example, 10-fold cross validation involves fitting ten models. This should be feasible in most cases. On the other hand, the leave-one-out scheme involves fitting N models (where N stands for the number of observations). This may be computationally very expensive and not always feasible.

We now try out **repeated 10-fold CV** to the same models and data as above where we used repeated half-sized CV. We again use the mean squared error (MSE) as performance measure.

```
set.seed(22)
##
mse.simple <- c()
mse.complex <- c()
##
for(i in 1:200){
  ## 1) prepare data
  ## randomly permute the rows:
  n <- nrow(d.trees)
  inds.permuted <- sample(1:n, replace = FALSE)
  d.trees.permuted <- d.trees[inds.permuted, ]

  ## create 10 (roughly) equally sized folds:
  ## K = number of folds
  K <- 10
  folds <- cut(1:n, breaks = K, labels = FALSE)
  ##
  ## perform K fold cross validation:
  mse.simple.per.fold <- integer(K)
  mse.complex.per.fold <- integer(K)

  for(k in 1:K){
    ## take the Kth fold as test set and the other folds as training set
    ind.test <- which(folds == k)
    d.trees.test <- d.trees.permuted[ind.test, ]
    d.trees.train <- d.trees.permuted[- ind.test, ]
    ##
    ## simple model ##
    ##
    ## 2) fit the model with "train" data
    lm.trees.1.train <- lm(formula = formula(lm.trees.1),
                          data = d.trees.train)
    ##
    ## 3) make prediction on the test data
    predicted.trees.1.test <- predict(lm.trees.1.train,
                                     newdata = d.trees.test)
    ##
    ## 4) compute MSE
    mse.simple.per.fold[k] <- mean((d.trees.test$growth.rate - predicted.trees.1.test)^2)
```

```
##
## complex model ##
##
## 2) fit the model with "train" data
lm.trees.2.train <- lm(formula = formula(lm.trees.2),
                        data = d.trees.train)
##
## 3) make prediction on the test data
predicted.trees.2.test <- predict(lm.trees.2.train,
                                  newdata = d.trees.test)
##
## 4) compute MSE
mse.complex.per.fold[k] <- mean((d.trees.test$growth.rate - predicted.trees.2.test)^2)
}

## we obtain the estimated test MSE for the 10-fold CV by averaging the
## test MSE on all 10 folds
mse.simple[i] <- mean(mse.simple.per.fold)
mse.complex[i] <- mean(mse.complex.per.fold)
}
```

We can now compare the mean MSE over the 200 repetitions for the two models.

```
mean(mse.simple)
```

```
[1] 0.066
```

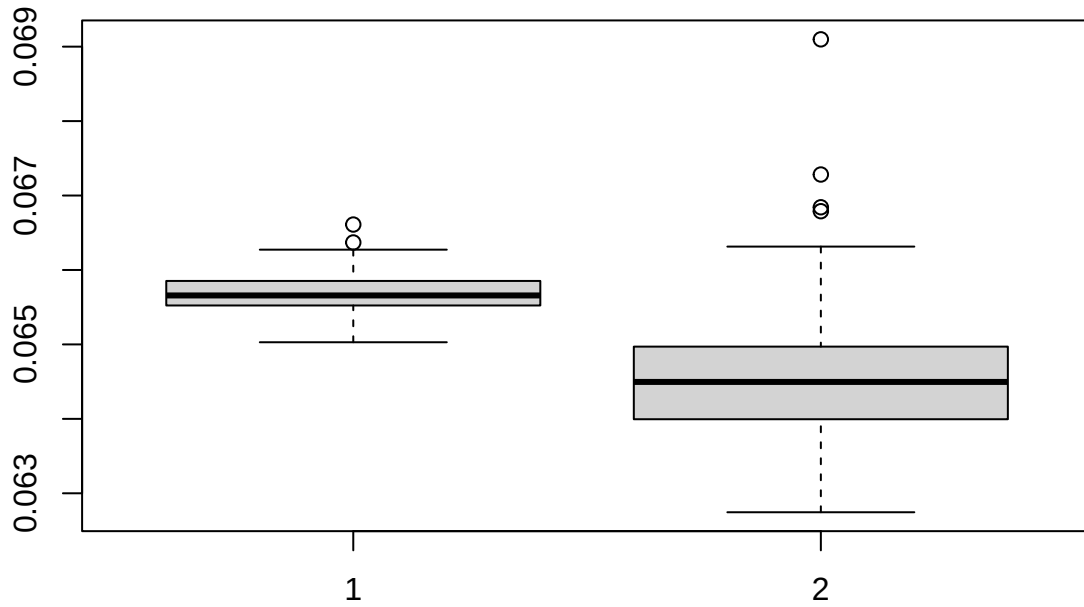
```
mean(mse.complex)
```

```
[1] 0.065
```

The results are even more similar as before, but this time the complex model seems slightly better.

We now visualise the results. That is we visualise the distribution of the estimated test MSE for the 200 different runs of 10-fold CV with a boxplot for both models.

```
boxplot(mse.simple, mse.complex)
```

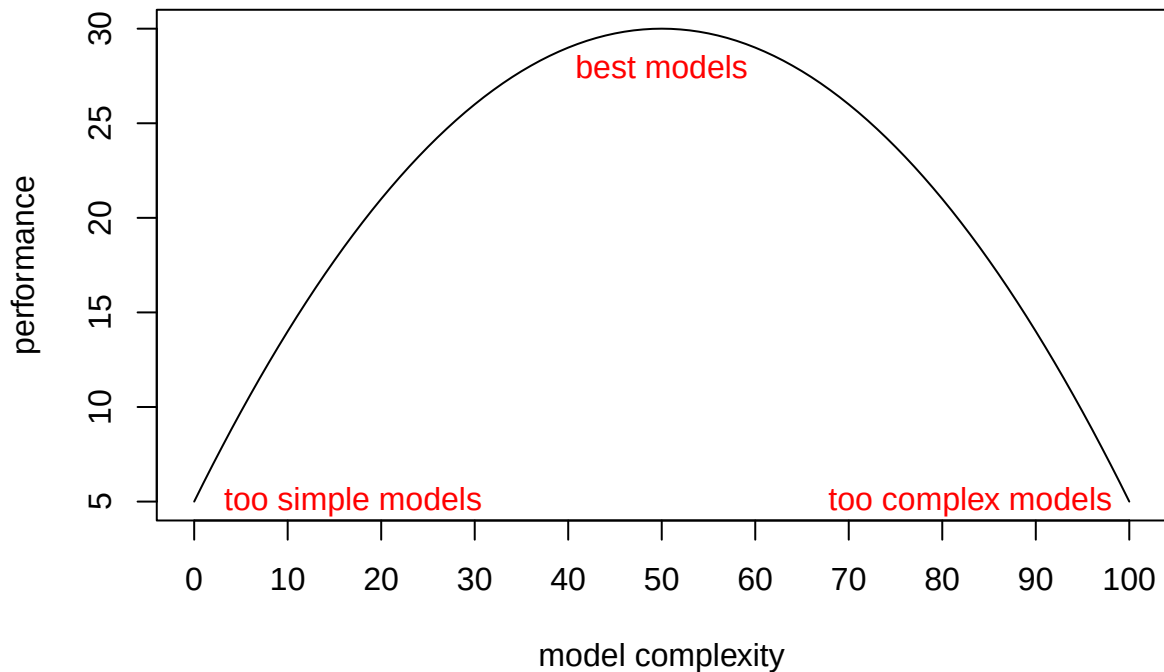


Based on this plot we would maybe deduce that the complex is better, because the MSE-estimates are lower in tendence for the complex model (lower MSE-values are better). However, note that the scale of the y-axis (the MSE-values) is very narrow and we would have to decide for a given setting if such a difference is relevant in practice at all. Generally, since both models seem to perform similarly out-of-sample, depending on the setting, the easier model could be selected (e.g. easier to communicate, less computational time etc.).

4 Take home messages

- when fitting a model we must decide when it is complex enough
- this decision can be based on cross validation
- cross validation is particularly suited when we want to optimise the predictive power of a model
- there are several schemes of cross validation
- there are several measures of fit that can be used to quantify the predictive performance of a model

In general, when using cross validation we usually aim at finding a model that maximises the predictive performance. A too simplistic model may miss some important features of the data and perform poorly. On the other hand, a too complex model may model noise in the data as being real structure and this also leads to a poor predictive performance. Graphically, this corresponds to the following graph.



Best models are... complex enough, but not too much.

Final note: In very many real settings we do not want to maximise the predictive performance of a model. For example, in the “doctor complaints” example above, we may want to understand which variables play a role (e.g. does sex play a role?) and quantify effects (e.g. how many more/less complaints do males get?). In this example, cross validation is not required.

On the other hand, if the “complaints model” was used to make predictions (e.g. to make predictions on the total number of complaints in a given hospital), then cross validation would be very helpful.

5 Appendix

5.1 What is “good”?

Ok, I fitted ten models and I get ten performances, the best being 0.25 MSE and the lowest being 0.5 MSE. Are my models good? The answer is “it depends”. Remember that MSE as many other performance measures cannot be judged in an absolute manner. A 0.25 MSE performance may be great in one field, and poor in another field of application.

What is **much more relevant in deciding whether a model is good is the definition of a benchmark**. If your model that has an MSE of 0.25 and will replace expertise-based predictions that have an MSE of 0.4 (computed from historical data) then we are doing well¹.

¹Obviously, the economic, social and so on costs of the change must be taken into account here, too.

Session Information

```
sessionInfo()
```

```
R version 4.5.1 (2025-06-13)
```

```
Platform: aarch64-apple-darwin20
```

```
Running under: macOS Sequoia 15.7.1
```

```
Matrix products: default
```

```
BLAS: /Library/Frameworks/R.framework/Versions/4.5-arm64/Resources/lib/libRblas.0.dylib
```

```
LAPACK: /Library/Frameworks/R.framework/Versions/4.5-arm64/Resources/lib/libRlapack.dylib; LAPACK vers
```

```
locale:
```

```
[1] en_US.UTF-8/en_US.UTF-8/en_US.UTF-8/C/en_US.UTF-8/en_US.UTF-8
```

```
time zone: Europe/Zurich
```

```
tzcode source: internal
```

```
attached base packages:
```

```
[1] stats      graphics  grDevices  utils      datasets  methods    base
```

```
other attached packages:
```

```
[1] faraway_1.0.9 knitr_1.50
```

```
loaded via a namespace (and not attached):
```

```
[1] lme4_1.1-37      digest_0.6.37    fastmap_1.2.0    Matrix_1.7-3
[5] xfun_0.52        lattice_0.22-7   splines_4.5.1    htmltools_0.5.8.1
[9] rmarkdown_2.29   tinytex_0.57     Rdpack_2.6.4     cli_3.6.5
[13] nloptr_2.2.1     grid_4.5.1       reformulas_0.4.1 compiler_4.5.1
[17] rbibutils_2.3    boot_1.3-31      rstudioapi_0.17.1 tools_4.5.1
[21] nlme_3.1-168     minqa_1.2.8      evaluate_1.0.4   Rcpp_1.0.14
[25] yaml_2.3.10      rlang_1.1.6      MASS_7.3-65
```